

EXP NO – 17

Counting Semaphore in SystemC

AIM

To design and simulate a **Counting Semaphore using SystemC** in EDA, and demonstrate how multiple processes access a limited number of shared resources.

PROGRAM

```
#include <systemc.h>

SC_MODULE(CountingSemaphore)
{
    sc_semaphore sem;

    SC_CTOR(CountingSemaphore) : sem(2)
    {
        SC_THREAD(process1);
        SC_THREAD(process2);
        SC_THREAD(process3);
    }

    void process1()
    {
        cout << sc_time_stamp() << " : Process 1 waiting for resource" << endl;

        sem.wait();

        cout << sc_time_stamp() << " : Process 1 acquired resource" << endl;
        wait(5, SC_NS);

        cout << sc_time_stamp() << " : Process 1 released resource" << endl;
        sem.post();
    }

    void process2()
    {
```

```

    cout << sc_time_stamp() << " : Process 2 waiting for resource" << endl;

    sem.wait();

    cout << sc_time_stamp() << " : Process 2 acquired resource" << endl;
    wait(8, SC_NS);

    cout << sc_time_stamp() << " : Process 2 released resource" << endl;
    sem.post();
}

void process3()
{
    cout << sc_time_stamp() << " : Process 3 waiting for resource" << endl;

    sem.wait();

    cout << sc_time_stamp() << " : Process 3 acquired resource" << endl;
    wait(4, SC_NS);

    cout << sc_time_stamp() << " : Process 3 released resource" << endl;
    sem.post();
}
};

int sc_main(int argc, char* argv[])
{
    CountingSemaphore cs("CountingSemaphore");

    sc_start(20, SC_NS);

    return 0;
}

```

SIMULATION :

The screenshot shows the EDA Playground web interface. The top navigation bar includes 'New', 'Run', and 'Save' buttons, along with a 'KnowHow WEBINARS' banner for 'Edge Machine Learning - Tips & Tricks' on 'AUG 7, 2026'. The left sidebar contains 'Languages & Libraries' with 'Testbench + Design' set to 'C++/SystemC' and 'Libraries' set to 'SystemC 2.3.3'. The main editor displays a C++ code snippet for a semaphore demo. The output window at the bottom shows the compilation and execution results.

```
1 #include <systemc.h>
2
3 SC_MODULE(BinarySemaphoreDemo)
4 {
5     sc_semaphore sem;
6
7     SC_CTOR(BinarySemaphoreDemo) : sem(1)
8     {
9         SC_THREAD(process1);
10        SC_THREAD(process2);
11    }
12
13    void process1()
14    {
15        cout << sc_time_stamp() << " : Process 1 waiting for semaphore" << endl;
16        sem.wait();
17        cout << sc_time_stamp() << " : Process 1 entered critical section" << endl;
18    }
19 }
```

SystemC 2.3.3-Accellera --- May 23 2020 14:33:56
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED
0 s : Process 1 waiting for semaphore
0 s : Process 1 entered critical section

OUTPUT

The screenshot shows the output window of the EDA Playground. It displays the compilation and execution results of the C++ code. The output includes the SystemC version, copyright information, and the execution timeline of the semaphore demo.

```
[2026-08-10 04:33:22 UTC] g++ -o sim -lsystemc *.cpp && echo "Compile done. Starting run..." && ./sim
Compile done. Starting run...

SystemC 2.3.3-Accellera --- May 23 2020 14:33:56
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED
0 s : Process 1 waiting for resource
0 s : Process 1 acquired resource
0 s : Process 2 waiting for resource
0 s : Process 2 acquired resource
0 s : Process 3 waiting for resource
5 ns : Process 1 released resource
5 ns : Process 3 acquired resource
8 ns : Process 2 released resource
9 ns : Process 3 released resource
Done
```

